

FitForm — Submission Notes

Google Drive Link

An AI fitness coach that watches your reps through the phone camera, calls out form mistakes in real time, and generates the next 3-day training block from the metrics it just observed. Built in Flutter, runs on Android.

1. What the app does

Two supported exercises: **Squat** and **Bicep Curl**.

End-to-end loop per session:

1. User signs in (Supabase Auth). First-time users walk through a 6-step biometric survey (gender, age, height, weight, goal, activity level) with drag-wheel interactions, stored locally and injected into every AI prompt.
2. They pick an exercise from the home carousel or the Courses list, see a detail screen with description + Time / Burn pills, and tap **Let's Workout**.
3. The camera opens, prompts for an open-hand gesture (held for 3 s) to start the set — no taps once you're in position.
4. ML Kit extracts 33 body landmarks per frame; the joint angle (knee for squats, elbow for curls) drives a live form gauge, a hardened rep counter, and corrective TTS cues ("Go deeper", "Squeeze at the top").
5. After each set, Gemini streams a short prose coaching tip into a bottom sheet, then a 30-second rest countdown leads into the next set.
1. After all 3 sets, Gemini's JSON-mode model emits a strict-schema 3-day adaptive plan with computed dates ("Wed, Jun 3"), times ("08:00 AM"), exercises, volumes, and per-day coaching intensity notes. The plan is parsed and rendered by a server-driven UI factory.

2. Tech stack & why

Layer	Choice	Reasoning
UI	Flutter (Dart)	Single codebase, fast iteration, excellent camera plugin
State	Riverpod (AsyncNotifier, StateProvider)	Clean separation of async lifecycle + testable controllers
Routing	GoRouter with ShellRoute	Auth-aware redirect, type-safe extra: payloads, full-screen routes for camera
Auth + DB	Supabase	Email/password auth out of the box; future-proof for session logging
Pose detection	google_mlkit_pose_detection	On-device, 33-landmark model, free, no cloud round-trip
AI	Google Generative AI (Gemini Flash)	Streaming + JSON-mode for the server-driven plan schema
TTS	flutter_tts	Built-in, no API key, sub-200ms latency
Local storage	shared_preferences	Simple K/V for biometrics + the latest plan JSON

Layer	Choice	Reasoning
Typography	google_fonts (Outfit)	One-line theming, no asset shipping

3. Architecture

```

lib/
├── app.dart                # MaterialApp.router + theme wiring
├── main.dart              # Supabase + dotenv bootstrap
├── core/
│   ├── constants/        # app_colors, app_text_styles, app_theme
│   └── providers/         # theme_mode_provider (light/dark toggle)
├── router/
│   └── app_router.dart    # GoRouter + auth-aware redirect
├── services/
│   ├── gemini_service.dart # Single source for Gemini calls (text + JSON)
│   └── supabase_service.dart
├── features/
│   ├── auth/             # Login / Sign-up views + shared AuthScaffold
│   ├── home/             # Greeting + carousel + AI panel
│   ├── profile/          # Biometric survey + summary
│   ├── plan/             # AI-generated plan rendering
│   │   ├── controllers/  # PlanController (AsyncNotifier)
│   │   └── widgets/      # dynamic_ui_parser.dart (server-driven factory)
│   ├── train/            # Courses list, exercise detail, camera + ML
│   ├── session/          # Post-session summary
│   └── shell/            # Bottom nav with centered launcher

```

Three-layer rule applied consistently:

- **Services** know the outside world (Supabase, Gemini, ML Kit). No widgets, no BuildContext.
- **Controllers / providers** orchestrate async state and persist to SharedPreferences.
- **Views** read state via ConsumerWidget / ref.watch; no direct service calls from inside a build.

4. Key design decisions

4.1 Server-driven UI for the AI plan

GeminiService.streamSessionAnalysis runs in JSON mode (responseMimeType: 'application/json') and returns a strict schema:

```

{
  "type": "WorkoutPlanView",
  "title": "...",
  "coachNote": "...",
  "sessions": [
    { "type": "SessionCard", "date": "Wed, Jun 3", "time": "08:00 AM", ... }
  ]
}

```

dynamic_ui_parser.dart is a Widget Function(Map<String, dynamic>, BuildContext) that switches on jsonNode['type'] and returns the right Flutter widget. The LLM is effectively *describing* the UI declaratively, and the app is a thin renderer. New node types can be added without redeploying.

4.2 Hardened rep counter (three stacked guards)

Naive angle-threshold counting fires false reps whenever someone walks past the camera. The current state machine rejects them with three independent guards layered on top of each other:

2. **Landmark confidence** — every joint used for the angle must have ML Kit likelihood $\geq$ `exercise.minLandmarkConfidence` (default 0.55). A weak frame produces a null angle and is dropped entirely.
3. **Strict crossing angles** — `repBottomAngleStrict` and `repTopAngleStrict` are *tighter* than the form-zone, so idle bending of the knee doesn't trip the state machine.
4. **Minimum phase duration** — once the machine enters down or up, the opposite phase cannot fire until `minRepDurationMs` has elapsed (default 800 ms). This kills sub-second jitter.

All four tunables live as fields on `Exercise` in `lib/features/train/models/exercise.dart`, so accuracy can be retuned without touching camera or pose code.

4.3 Gesture-based start

After pose detection is up, the app waits for an open-hand pose held for 3 seconds — detected by checking that thumb/index/pinky are all above the wrist landmark. Removes the need to tap a "Start" button while you're getting into position.

4.4 Aspect-ratio-safe camera

`CameraPreview` inside a `Stack(StackFit.expand)` distorts the feed because the phone screen is ~19.5:9 and the sensor is ~4:3. Wrapping it in `FittedBox(BoxFit.cover) + SizedBox(sensorSize)` preserves the native ratio and crops the sides — same approach Instagram and TikTok use.

4.5 Single theme with explicit light + dark

`AppTheme.light` and `AppTheme.dark` both flow through one `_build()` factory in `app_theme.dart`. Dark uses lime accent (#C8FF2D), light uses orange (#FF9A2E). A home-screen icon flips a `StateProvider<ThemeMode>` so the design can be previewed in both modes during dev without rebuilding.

5. Challenges & how I solved them

Challenge	Resolution
Rep counter false positives while standing or walking past the camera.	Added three independent guards (confidence, strict angles, min phase duration) instead of fighting one threshold at a time.
Camera feed stretched vertically on the 19.5:9 device.	Replaced the naive <code>StackFit.expand</code> preview with a <code>FittedBox(cover)</code> wrapper sized to the sensor's native preview size.
Gemini occasionally returning markdown-wrapped JSON.	Switched to <code>GenerationConfig(responseMimeType: 'application/json')</code> which forces raw JSON; the controller's <code>jsonDecode</code> is wrapped in a <code>try/catch</code> that surfaces failures as <code>AsyncValue.error</code> instead of crashing.

Challenge	Resolution
First-time users landing on an empty Plan tab.	Seeded a starter plan in SharedPreferences on first launch + added the biometric survey redirect from Home so the app is never empty.
Static dates ("tomorrow", "next day") in early Gemini outputs.	Pass <code>DateTime.now()</code> to the service and pre-compute Day 1/2/3 anchors in the prompt — the model now emits exact dates like "Wed, Jun 3" that the UI renders directly.
Stretched / awkward auth screens.	Built a shared AuthScaffold with a ClipPath + custom CustomClipper<Path> diagonal divider between the hero image and the form panel; one component reused by both Login and Sign-up.
Inconsistent dark/light styling drift.	Centralised every color and text style through <code>Theme.of(context).colorScheme / textTheme</code> . Legacy <code>AppColors.background</code> etc. kept as dark-mode aliases for older widgets.

6. How to run

You have two options. **Option A is the fastest path for an assessor** — just install the prebuilt APK on an Android device. Option B is for anyone who wants to read + run the source. Option C is just download the link provided in the google drive link.

Option A — Install the prebuilt APK (~2 min)

Pre-configured with the Supabase and Gemini keys baked in. No `.env` setup required.

- Download the APK from the GitHub Releases tab of this repo (*or use the link shared in the submission email*).
- On the Android device: **Settings → Apps → Special access → Install unknown apps** → allow your browser / file manager.
- Open the `.apk` file, tap **Install**.
- Launch **FitForm**, grant camera permission when prompted.

Note: *the embedded keys are tied to a sandbox Supabase project and will be rotated immediately after assessment.*

Option B — Run from source

Prerequisites

- Flutter SDK ≥ 3.38 (Dart 3.10+)
- Android Studio / Android SDK + a physical device or emulator
- A Supabase project (URL + anon key)
- A Gemini API key — aistudio.google.com/app/apikey

One-time setup

```
git clone <repo-url>
cd fitness
flutter pub get
cp .env.example .env          # then fill in real values
```

.env is gitignored — populate it with your own keys:

```
SUPABASE_URL=https://<your-project>.supabase.co
SUPABASE_ANON_KEY=<your-anon-key>
GEMINI_API_KEY=<your-gemini-key>
```

In the Supabase dashboard under **Authentication → Providers → Email**, toggle **"Confirm email" off** so sign-up returns a session immediately (otherwise the confirmation link's default redirect points at localhost).

Drop your hero photos into assets/images/:

```
assets/images/
├── squat.png          # squat hero (home carousel, train list, detail)
├── bicep.png          # bicep curl hero
└── auth_hero.png      # login + sign-up background (optional, has fallback)
```

Run

```
flutter run          # connects to the first attached device
```

Release build (produces the APK shipped in Option A)

```
flutter build apk --release
# output: build/app/outputs/flutter-apk/app-release.apk
```

First-launch behavior

9. Land on Login → sign up with email + password.
10. Routed to the **6-step biometric survey** (gender, age, height, weight, goal, activity level). Save flips to Home.
11. Home shows the seeded starter plan; the AI Personalised Plan panel routes to the Plan tab.
12. Open Train → tap Squat or Bicep Curl → tap **Let's Workout** → grant camera permission → show open hand to start counting reps.

7. What I'd improve with more time

In rough priority order:

13. **Supabase session logging.** Tables for sessions + sets, write on workout end, read last N sessions into Gemini's prompt for true progress awareness.
14. **Performance dashboard on Home.** Recent sessions list, weekly rep chart, streak counter — currently the plan is the only feedback surface.
15. **ROM (Range-of-Motion) per rep** + a composite TUT × ROM "Endurance Bar" to replace the static form gauge.
16. **Side auto-detection.** Currently hardcoded to left-side landmarks. Pick whichever limb has higher .likelihood so users can face either way.
17. **Off-camera grace.** If landmarks disappear mid-set, pause the timer + rep counter and prompt "Step into frame" instead of silently freezing.
18. **TTS audio ducking** so music apps drop their volume during cues, plus a voice-pack picker (drill sergeant vs. encouraging coach).

19. **Onboarding overlay** showing camera placement + recommended distance the first time the camera screen opens.
20. **Test coverage.** Unit tests for the rep state machine + golden tests for the dynamic UI parser are the highest-value targets.